

Exploratory Data Analysis (EDA)

Complete 16-Mark Answers with Python Code, Explanations & Optimal Solutions

Question 1 (16 Marks) – Customer Purchase Data

Dataset: Customer_ID, City, Category, Amount. Tasks: (a) Group by City – total purchase (b) Aggregate amount category-wise (c) Pivot table City vs Category (d) Explain how transformed data supports EDA.

Step 0 – Create the DataFrame

```
import pandas as pd
import numpy as np

data = {
    'Customer_ID': ['C101', 'C102', 'C103', 'C104', 'C105', 'C106'],
    'City': ['Chennai', 'Coimbatore', 'Chennai', 'Madurai', 'Coimbatore', 'Chennai'],
    'Category': ['Electronics', 'Clothing', 'Electronics', 'Furniture', 'Clothing', 'Furniture'],
    'Amount': [45000, 12000, 30000, 25000, 18000, 22000]
}
df = pd.DataFrame(data)
print(df)
```

Explanation: We first construct a pandas DataFrame from the given tabular data. All subsequent EDA operations (groupby, pivot, aggregation) are performed on this DataFrame.

(a) Group by City – Total Purchase Amount

```
city_total = df.groupby('City')['Amount'].sum().reset_index()
city_total.columns = ['City', 'Total_Amount']
city_total = city_total.sort_values('Total_Amount', ascending=False)
print(city_total)
```

Explanation: groupby('City') partitions rows by city; .sum() aggregates the Amount column. reset_index() converts the result back to a regular DataFrame. Sorting reveals the ranking immediately.

Output:

City	Total_Amount
Chennai	97000
Coimbatore	30000
Madurai	25000

Insight: Chennai is the dominant market (Rs.97,000 ≈ 64% of total revenue).

(b) Aggregate Amount Category-wise

```
cat_total = df.groupby('Category')['Amount'].agg(['sum', 'mean', 'count'])
cat_total.columns = ['Total_Amount', 'Avg_Amount', 'Num_Transactions']
cat_total = cat_total.sort_values('Total_Amount', ascending=False)
print(cat_total)
```

Explanation: Using .agg() we compute multiple statistics in one pass – total revenue, average transaction value and number of transactions per category. This richer summary supports deeper EDA than a simple sum.

Output:

Category	Total_Amount	Avg_Amount	Num_Transactions
Electronics	75000	37500	2
Furniture	47000	23500	2
Clothing	30000	15000	2

(c) Pivot Table – City vs Category

```
pivot = pd.pivot_table(df, values='Amount', index='City',
                        columns='Category', aggfunc='sum',
                        fill_value=0, margins=True, margins_name='Total')
print(pivot)
```

Explanation: pivot_table reshapes the data so that Cities become rows and Categories become columns. aggfunc='sum' fills each cell with the total Amount. fill_value=0 replaces missing combinations with 0; margins=True adds row and column totals – extremely useful for

EDA.

Output:

City	Clothing	Electronics	Furniture	Total
Chennai	0	75000	22000	97000
Coimbatore	30000	0	0	30000
Madurai	0	0	25000	25000
Total	30000	75000	47000	152000

(d) How the Transformed Dataset Supports Exploratory Analysis

- Pattern discovery:** The pivot immediately shows that Electronics sales exist only in Chennai, Clothing only in Coimbatore, and Furniture in both Chennai and Madurai – combinations that are invisible in the raw row-wise data.
- Dimensionality reduction:** Six transaction rows collapse into a 3×3 summary, making visual inspection and ranking trivial.
- Ready for visualization:** The aggregated tables can be fed directly to `sns.barplot` / `sns.heatmap` without further preprocessing.
- Business questions answered:** “Which city–category pair should receive more inventory?” is answered at a glance (Chennai–Electronics).
- Scalability:** The same `groupby/pivot` code works unchanged on thousands or millions of rows, which is the practical value of aggregation in real-world EDA.

Question 2 (16 Marks) – Product & Sales Merge + Aggregation

Product dataset (Product_ID, Product, Category) + Sales dataset (Product_ID, Region, Sales). Tasks: Merge, Group by Category & Region, Aggregate total sales, Construct pivot table.

Step 0 – Create both DataFrames

```
product_df = pd.DataFrame({
    'Product_ID': ['P101', 'P102', 'P103'],
    'Product': ['Laptop', 'Chair', 'Shirt'],
    'Category': ['Electronics', 'Furniture', 'Clothing']
})
sales_df = pd.DataFrame({
    'Product_ID': ['P101', 'P102', 'P103', 'P101'],
    'Region': ['North', 'South', 'North', 'South'],
    'Sales': [150000, 45000, 25000, 120000]
})
```

(a) Merge the datasets on Product_ID

```
merged = pd.merge(product_df, sales_df, on='Product_ID', how='inner')
print(merged)
```

Explanation: `pd.merge` performs a relational join. `how='inner'` keeps only matching Product_IDs. The result contains product attributes (name, category) together with every regional sales record. Alternative joins (left/right/outer) can be chosen if unmatched rows must be retained.

Output:

Product_ID	Product	Category	Region	Sales
P101	Laptop	Electronics	North	150000
P101	Laptop	Electronics	South	120000
P102	Chair	Furniture	South	45000
P103	Shirt	Clothing	North	25000

(b) & (c) Group by Category and Region + Total Sales

```
grouped = (merged.groupby(['Category', 'Region'])['Sales']
            .sum()
            .reset_index()
            .sort_values('Sales', ascending=False))
print(grouped)
```

Explanation: Multi-column `groupby` creates a hierarchical partition. Summing Sales inside each (Category, Region) cell produces the required aggregation. Sorting by Sales ranks the combinations.

Output:

Category	Region	Sales
Electronics	North	150000
Electronics	South	120000
Furniture	South	45000
Clothing	North	25000

(d) Pivot Table Summarizing Results

```
pivot = pd.pivot_table(merged, values='Sales', index='Category',
                        columns='Region', aggfunc='sum',
                        fill_value=0, margins=True)

print(pivot)
```

Explanation: The pivot table converts the long grouped result into a matrix form that is ideal for heatmaps and for quick cross-regional comparison. margins=True supplies category totals and grand total automatically.

Output:

Category	North	South	All
Clothing	25000	0	25000
Electronics	150000	120000	270000
Furniture	0	45000	45000
All	175000	165000	340000

Key insight: Electronics contributes 79% of total revenue (2,70,000 / 3,40,000). Furniture and Clothing are region-specific. The pivot is ready for `sns.heatmap(pivot)` visualization.

Question 3 (16 Marks) – Temperature Dataset Analysis

Month-wise temperature (°C) Jan–Aug. Tasks: Select visualizations; Analyse seasonal patterns; Role of labels/legends/annotations; Evaluate histogram suitability.

Step 0 – Data + Imports

```
import matplotlib.pyplot as plt
import seaborn as sns

temp_df = pd.DataFrame({
    'Month': ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug'],
    'Temperature': [28, 30, 33, 36, 38, 35, 32, 31]
})
```

(a) Appropriate Visualizations (with code)

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# 1. Line plot – best for sequential trend
axes[0].plot(temp_df['Month'], temp_df['Temperature'],
             marker='o', color='crimson', linewidth=2, markersize=7)
axes[0].set_title('Monthly Temperature Trend', fontsize=12, fontweight='bold')
axes[0].set_xlabel('Month')
axes[0].set_ylabel('Temperature (°C)')
axes[0].axhline(y=temp_df['Temperature'].mean(), color='gray',
               linestyle='--', label=f"Mean = {temp_df['Temperature'].mean():.1f}°C")
axes[0].legend()
axes[0].annotate('Peak 38°C', xy=('May', 38), xytext=('Jun', 39),
               arrowprops=dict(arrowstyle='->', color='black'), fontsize=9)

# 2. Bar chart – absolute comparison
axes[1].bar(temp_df['Month'], temp_df['Temperature'], color='steelblue', edgecolor='black')
axes[1].set_title('Temperature by Month (Bar Chart)', fontsize=12, fontweight='bold')
axes[1].set_xlabel('Month')
axes[1].set_ylabel('Temperature (°C)')

plt.tight_layout()
plt.show()
```

Explanation: A line plot is optimal because temperature is ordered in time; the continuous line reveals the rise-and-fall pattern. A bar chart is a useful secondary view for comparing discrete monthly values. Both plots are created side-by-side with subplots for efficient comparison.

(b) Seasonal Pattern Analysis

Temperature rises steadily from January (28°C) to a clear peak in May (38°C) – a classic pre-monsoon heating pattern. From June onwards temperatures decline (35→31°C), consistent with the onset of the monsoon / cooler season. The peak-to-trough drop of 7°C is immediately visible on the annotated line plot. No secondary peaks appear, indicating a single seasonal cycle in the given eight-month window.

(c) Role of Labels, Legends and Annotations

- **Title & axis labels** tell the reader exactly what variable is plotted and its unit (°C).
- **Legend** identifies the horizontal mean line so the reader can judge which months lie above or below average.
- **Annotation** (arrow + text at the May peak) draws attention to the most important data point without forcing the reader to scan every value. Together these elements turn a raw plot into a self-explanatory analytical graphic.

(d) Are Histograms Suitable?

```
# Histogram – possible but NOT optimal for this data
plt.figure(figsize=(5, 3))
plt.hist(temp_df['Temperature'], bins=5, color='teal', edgecolor='black')
plt.title('Temperature Distribution (Histogram)')
plt.xlabel('Temperature (°C)')
plt.ylabel('Frequency')
plt.show()
```

A histogram shows the frequency distribution of temperature values and completely discards the temporal order. Because the scientific interest lies in the seasonal progression (order matters), a histogram is **not suitable as the primary visualization**. It may be used secondarily to examine overall spread, but the line/bar plots remain far superior for this ordered dataset.

Question 4 (16 Marks) – Customer Age Distribution

Ages: 22, 26, 31, 35, 40, 45. Tasks: Visualize distribution; Numerical summaries; Seaborn usefulness; Justify chosen method.

Step 0 – Data

```
age_df = pd.DataFrame({
    'Customer': ['C1', 'C2', 'C3', 'C4', 'C5', 'C6'],
    'Age': [22, 26, 31, 35, 40, 45]
})
```

(a) Visualizing the Age Distribution

```
fig, axes = plt.subplots(1, 3, figsize=(14, 4))

# Histogram + KDE
sns.histplot(age_df['Age'], bins=4, kde=True, ax=axes[0], color='royalblue')
axes[0].set_title('Histogram + KDE')

# Box plot
sns.boxplot(y=age_df['Age'], ax=axes[1], color='lightgreen')
axes[1].set_title('Box Plot')

# Swarm / strip plot (shows every point)
sns.swarmplot(y=age_df['Age'], ax=axes[2], color='darkorange', size=8)
axes[2].set_title('Swarm Plot (individual points)')

plt.suptitle('Age Distribution – Multiple Views', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()
```

Explanation: With only six observations we use three complementary views. The histogram+KDE shows shape and density; the box plot summarises median/IQR; the swarm plot displays every individual age so no information is hidden by binning.

(b) Numerical Summaries

```
print(age_df['Age'].describe())
print('Range :', age_df['Age'].max() - age_df['Age'].min())
print('Skew :', round(age_df['Age'].skew(), 3))
```

Output of `.describe()`: count=6, mean≈33.17, std≈8.61, min=22, 25%=27.25, 50%(median)=33, 75%=38.75, max=45. Range=23 years. Skewness ≈ 0.14 (nearly symmetric). No outliers are present (all points lie within 1.5×IQR of the quartiles).

(c) Usefulness of Seaborn

Seaborn's high-level API (histplot, boxplot, swarmplot, violinplot) produces statistically informed, aesthetically pleasing graphics with a single function call. It automatically manages binning, kernel-density estimation, quartile calculation and colour themes. For rapid EDA this is far more productive than writing equivalent Matplotlib code from scratch.

(d) Justification of Chosen Method

The combination of histogram+KDE + box plot + swarm plot is optimal because: (1) histogram reveals distributional shape, (2) box plot gives a compact five-number summary, (3) swarm plot guarantees that every data point remains visible – critical when n is small. Together they satisfy both statistical and visual completeness for exploratory analysis of a small numeric sample.

Question 5 (16 Marks) – Student Marks (3 Subjects)

Students S1–S6, marks in Python / Statistics / EDA. Tasks: Best viz for Python distribution; Compare three subjects; Role of legends/colors/labels/annotations; Seaborn vs Matplotlib.

Step 0 – Data

```
marks = pd.DataFrame({
    'Student': ['S1', 'S2', 'S3', 'S4', 'S5', 'S6'],
    'Python': [75, 85, 62, 90, 70, 82],
    'Statistics': [68, 80, 70, 85, 65, 78],
    'EDA': [72, 88, 65, 92, 74, 85]
})
```

(a) Most Suitable Visualization for Python Marks Distribution

```
fig, ax = plt.subplots(figsize=(6, 4))
sns.histplot(marks['Python'], bins=4, kde=True, color='steelblue', edgecolor='black')
ax.axvline(marks['Python'].mean(), color='red', linestyle='--',
           label=f"Mean = {marks['Python'].mean():.1f}")
ax.axvline(marks['Python'].median(), color='green', linestyle='-.',
           label=f"Median = {marks['Python'].median():.1f}")
ax.set_title('Distribution of Python Marks')
ax.set_xlabel('Marks')
ax.legend()
plt.show()
```

Justification: A histogram with KDE is ideal for examining the distribution of a single continuous variable. It shows modality, spread and central tendency. With only six scores, 3–5 bins keep the shape readable. Mean and median lines further aid interpretation. A box plot is an equally valid alternative when the five-number summary is the primary interest.

(b) Comparing Performance of the Three Subjects

```
# Long format for Seaborn
long = marks.melt(id_vars='Student', var_name='Subject', value_name='Marks')

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# Grouped bar chart
sns.barplot(data=long, x='Student', y='Marks', hue='Subject', ax=axes[0])
axes[0].set_title('Marks by Student & Subject')
axes[0].legend(title='Subject')

# Box plots side-by-side
sns.boxplot(data=long, x='Subject', y='Marks', ax=axes[1], palette='Set2')
axes[1].set_title('Subject-wise Distribution')

plt.tight_layout()
plt.show()

# Also useful: correlation heatmap
sns.heatmap(marks[['Python', 'Statistics', 'EDA']].corr(),
            annot=True, cmap='coolwarm', vmin=-1, vmax=1)
plt.title('Subject Correlation Heatmap')
plt.show()
```

Explanation: melt() converts the wide table into long (tidy) form required by most Seaborn categorical plots. The grouped bar chart compares subjects within each student; the box plots compare overall distributions. The correlation heatmap quantifies linear relationships among subjects. From the data, S4 is the strongest overall performer; Statistics tends to score slightly lower than Python and EDA.

(c) Improving Interpretation with Visual Elements

- **Colors** (hue / palette) consistently map each subject to a distinct colour across all plots.
- **Legend** translates colour into subject name – essential for multi-series plots.
- **Axis labels & titles** remove any ambiguity about units and what is being compared.
- **Annotations** (mean lines, value labels on bars, or text call-outs for highest/lowest scores) highlight the key takeaway without requiring the reader to inspect every number.

(d) Seaborn vs Matplotlib – Choice & Justification

Seaborn is the more appropriate choice for this analysis. Reasons: (1) One-line statistical plots (boxplot, histplot, heatmap, barplot with automatic aggregation); (2) Built-in themes and colour palettes that already look professional; (3) Native support for long-form data and the hue semantic, which drastically reduces boilerplate. Matplotlib is still used underneath (and for fine layout control via subplots), but the exploratory cycle is significantly faster with Seaborn.

Question 6 (16 Marks) – Retail Store Sales Analysis

Product, Category, Region, Quantity, Sales. Tasks: Group by Category (Sales & Quantity); Pivot Category × Region; Highest-selling category; Business decision support.

Step 0 – Data

```
retail = pd.DataFrame({
    'Product': ['Laptop', 'Mobile', 'Chair', 'Table', 'Shirt', 'Jeans'],
    'Category': ['Electronics', 'Electronics', 'Furniture', 'Furniture', 'Clothing', 'Clothing'],
    'Region': ['North', 'South', 'North', 'South', 'North', 'South'],
    'Quantity': [5, 10, 8, 4, 12, 10],
    'Sales': [250000, 180000, 80000, 60000, 36000, 30000]
})
```

(a) Group by Category – Total Sales & Quantity

```
cat_agg = (retail.groupby('Category')
            .agg(Total_Quantity=('Quantity', 'sum'),
                Total_Sales=('Sales', 'sum'),
                Avg_Sales=('Sales', 'mean'))
            .sort_values('Total_Sales', ascending=False)
            .reset_index())

print(cat_agg)
```

Explanation: Named aggregation (new in recent pandas) produces clearly labelled columns in one step. Sorting by Total_Sales ranks categories by revenue importance.

Output:

Category	Total_Quantity	Total_Sales	Avg_Sales
Electronics	15	430000	215000
Furniture	12	140000	70000
Clothing	22	66000	33000

(b) Pivot Table – Category-wise Sales by Region

```
pivot = pd.pivot_table(retail, values='Sales', index='Category',
                        columns='Region', aggfunc='sum',
                        fill_value=0, margins=True)

print(pivot)
```

Output:

Category	North	South	All
Clothing	36000	30000	66000
Electronics	250000	180000	430000
Furniture	80000	60000	140000
All	366000	270000	636000

(c) Highest-Selling Category

```
highest = cat_agg.iloc[0] # already sorted
print(f"Highest-selling category: {highest['Category']} "
      f"with Total Sales = Rs.{highest['Total_Sales']:,}")
```

Electronics is the highest-selling category (Rs.4,30,000). Note that Clothing leads in quantity sold (22 units) but generates the lowest revenue – an important distinction between volume and value.

(d) Support for Business Decision-Making

- Allocate more shelf space / marketing budget to Electronics, especially in the North (Rs.2,50,000).
- Investigate Clothing's high volume / low revenue – possible pricing or product-mix opportunity.
- Regional disparity (North 57% of sales) can guide expansion or targeted promotions in the South.
- The aggregated and pivoted tables become the data source for executive dashboards and further what-if analyses.

Question 7 (16 Marks) – Multi-level Sales Aggregation

Product, Region, Quantity, Sales. Tasks: Group by Region; Group by Product; Pivot Product × Region; Further EDA use of results.

Step 0 – Data

```
sales = pd.DataFrame({
    'Product': ['Laptop', 'Mobile', 'Laptop', 'Tablet', 'Mobile', 'Tablet'],
    'Region': ['North', 'South', 'South', 'North', 'North', 'South'],
    'Quantity': [5, 10, 3, 8, 6, 4],
    'Sales': [250000, 180000, 150000, 120000, 108000, 60000]
})
```

(a) Group by Region – Total Sales

```
region_sales = (sales.groupby('Region')['Sales']
                .sum()
                .reset_index()
                .sort_values('Sales', ascending=False))

print(region_sales)
```

Output: North = 4,78,000 ; South = 3,90,000. North leads.

(b) Group by Product – Total Quantity & Sales

```
prod_agg = (sales.groupby('Product')
            .agg(Total_Qty=('Quantity', 'sum'),
                 Total_Sales=('Sales', 'sum'))
            .sort_values('Total_Sales', ascending=False)
            .reset_index())
# Optional: average selling price
prod_agg['ASP'] = prod_agg['Total_Sales'] / prod_agg['Total_Qty']
print(prod_agg)
```

Explanation: Computing Average Selling Price (ASP = Sales/Qty) adds a value-per-unit metric that pure totals cannot reveal. Laptop has the highest ASP; Mobile has the highest volume.

Output:

Product	Total_Qty	Total_Sales	ASP
Laptop	8	400000	50000
Mobile	16	288000	18000
Tablet	12	180000	15000

(c) Pivot Table – Product-wise Sales by Region

```
pivot = pd.pivot_table(sales, values='Sales', index='Product',
                        columns='Region', aggfunc='sum',
                        fill_value=0, margins=True)

print(pivot)
```

Output:

Product	North	South	All
Laptop	250000	150000	400000
Mobile	108000	180000	288000
Tablet	120000	60000	180000
All	478000	390000	868000

(d) Further Exploratory Analysis Enabled by These Results

The aggregated tables become the foundation for: ranked bar charts, stacked regional contribution charts, ASP comparison plots, and identification of weak product–region cells (e.g. Tablet–South). They also feed directly into correlation or clustering analyses if more attributes are later joined. In short, aggregation converts raw transactional data into decision-ready summaries.

Question 8 (16 Marks) – Student & Marks Merge + Reshape

Student (ID, Name, Dept) + Marks (ID, Python, Statistics). Tasks: Merge; Reshape; Average marks by Department; Cross-tab Department vs performance category.

Step 0 – DataFrames

```
student_df = pd.DataFrame({
    'Student_ID': [101, 102, 103, 104],
    'Name': ['Arun', 'Banu', 'Charan', 'Divya'],
    'Department': ['AI&DS', 'CSE', 'AI&DS', 'CSE']
})
marks_df = pd.DataFrame({
    'Student_ID': [101, 102, 103, 104],
    'Python': [85, 72, 91, 68],
    'Statistics': [78, 88, 84, 75]
})
```

(a) Merge on Student_ID

```
merged = pd.merge(student_df, marks_df, on='Student_ID', how='inner')
print(merged)
```

Explanation: Inner join on the common key produces a single analytic table containing demographic and performance attributes for every student.

(b) Reshape for Analysis


```
# Wide → Long (tidy) form - useful for Seaborn categorical plots
long = merged.melt(id_vars=['Student_ID', 'Name', 'Department'],
                  value_vars=['Python', 'Statistics'],
                  var_name='Subject', value_name='Marks')
print(long.head())
```

Explanation: melt() stacks the subject columns into a single “Marks” column and adds a “Subject” identifier. Many visualization and modelling functions prefer this long format. The original wide form is retained for subject-wise arithmetic (means, correlations).

(c) Average Python & Statistics Marks by Department

```
dept_avg = (merged.groupby('Department')[['Python', 'Statistics']]
            .mean()
            .round(1)
            .reset_index())
print(dept_avg)
```

Output:

Department	Python	Statistics
AI&DS;	88.0	81.0
CSE	70.0	81.5

AI&DS; outperforms CSE in Python; Statistics averages are almost identical.

(d) Cross-tabulation – Department vs Performance Category

```
# Create a simple performance label
merged['Performance'] = np.where(
    (merged['Python'] >= 80) & (merged['Statistics'] >= 80), 'High',
    np.where((merged['Python'] < 70) | (merged['Statistics'] < 70), 'Low', 'Medium')
)

ct = pd.crosstab(merged['Department'], merged['Performance'],
                margins=True, margins_name='Total')
print(ct)

# Optional: percentage view
ct_pct = pd.crosstab(merged['Department'], merged['Performance'],
                    normalize='index').round(2)*100
print(ct_pct)
```

Explanation: A derived categorical column (Performance) is created with clear business rules. pd.crosstab then builds a contingency table of Department × Performance. The normalize option converts counts to row-wise percentages, making departmental comparisons fair regardless of size. This is a classic EDA technique for exploring relationships between two categorical variables.

Question 9 / 10 (16 Marks) – Student Marks (8 Students)

Students A–H, marks in Python / Statistics / EDA. Tasks: Best viz for Python distribution + justification; Subplots for subject comparison; Visual elements; Usefulness of 3-D plot.

Step 0 – Data

```
df = pd.DataFrame({
    'Student': list('ABCDEFGH'),
    'Python':   [78, 85, 62, 91, 74, 55, 88, 69],
    'Statistics': [65, 72, 58, 88, 70, 60, 82, 64],
    'EDA':      [72, 80, 65, 94, 76, 58, 90, 70]
})
```

(a) Most Appropriate Visualization for Python Marks + Justification

```
fig, ax = plt.subplots(figsize=(6, 4))
sns.histplot(df['Python'], bins=5, kde=True, color='teal', edgecolor='black', ax=ax)
ax.axvline(df['Python'].mean(), color='red', ls='--', label=f"Mean={df['Python'].mean():.1f}")
ax.axvline(df['Python'].median(), color='orange', ls='-.', label=f"Median={df['Python'].median():.1f}")
ax.set_title('Distribution of Python Marks')
ax.set_xlabel('Marks')
ax.legend()
plt.show()

# Alternative / complementary: box plot
sns.boxplot(x=df['Python'], color='lightblue')
plt.title('Box Plot of Python Marks')
plt.show()
```

Justification: Histogram + KDE is the standard tool for examining the shape, centre and spread of a univariate continuous distribution. With $n=8$, five bins avoid both over- and under-smoothing. Adding mean/median lines and a complementary box plot gives a complete five-number + density view. These plots answer the classic EDA questions: Is the distribution skewed? Are there outliers? Where is the bulk of the data?

(b) Using Subplots to Compare the Three Subjects

```
fig, axes = plt.subplots(1, 3, figsize=(14, 4), sharey=True)

for ax, subject, color in zip(axes,
                               ['Python', 'Statistics', 'EDA'],
                               ['steelblue', 'darkorange', 'seagreen']):
    sns.histplot(df[subject], bins=5, kde=True, ax=ax, color=color, edgecolor='black')
    ax.set_title(f'{subject} Marks')
    ax.axvline(df[subject].mean(), color='red', ls='--', lw=1.2)

plt.suptitle('Subject-wise Mark Distributions (shared Y-axis)', fontsize=13)
plt.tight_layout()
plt.show()

# Alternative: side-by-side box / violin
long = df.melt(id_vars='Student', var_name='Subject', value_name='Marks')
sns.boxplot(data=long, x='Subject', y='Marks', palette='Set2')
plt.title('Comparative Box Plots')
plt.show()
```

Explanation: sharey=True forces a common vertical scale so differences in location and spread are immediately comparable. Colour-coding each subject and adding a mean line further aids reading. A single grouped box/violin plot is a compact alternative when space is limited.

(c) Legends, Colours, Titles and Annotations

- Consistent **colour** per subject across every plot builds visual memory.
- **Titles** (figure + subplot) orient the reader instantly.
- **Legend / mean-line labels** encode the additional reference statistics.
- **Annotations** (arrows pointing to outliers or text boxes with exact values) surface the most important observations without requiring the reader to hover or calculate.

Collectively these elements transform raw charts into self-contained analytical statements.

(d) Is a Three-Dimensional Plot Useful?

```
from mpl_toolkits.mplot3d import Axes3D
fig = plt.figure(figsize=(7, 5))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(df['Python'], df['Statistics'], df['EDA'],
           c=df['Python'], cmap='viridis', s=60)
ax.set_xlabel('Python')
ax.set_ylabel('Statistics')
ax.set_zlabel('EDA')
ax.set_title('3-D Scatter of Three Subjects')
plt.show()
```

A 3-D scatter can in principle display the joint relationship among three variables. However, for only eight points it is usually **less useful** than a pair-plot matrix or a correlation heatmap: perspective distortion, over-plotting and the difficulty of reading

exact coordinates make patterns harder to perceive. Recommended alternative:

```
sns.pairplot(df[['Python', 'Statistics', 'EDA']], diag_kind='hist')
plt.suptitle('Pair Plot – Preferred Multivariate View', y=1.02)
plt.show()

sns.heatmap(df[['Python', 'Statistics', 'EDA']].corr(),
            annot=True, cmap='coolwarm', center=0)
plt.title('Correlation Heatmap')
plt.show()
```

Pair plots and heatmaps remain the practical gold standard for multivariate EDA with small-to-medium n.

Question 11 (16 Marks) – E-commerce Transaction EDA

Customer transaction data (ID, product, category, price, quantity, purchase date). Tasks: Group/aggregate for category-wise sales; Suggest pivot structure; Matplotlib plots for transaction-value distribution; Seaborn for relationships.

(a) Grouping & Aggregating for Category-wise Sales

```
# Assume df already loaded with columns:
# customer_id, product, category, price, quantity, purchase_date

df['Sales'] = df['price'] * df['quantity']          # derived metric
df['Month'] = pd.to_datetime(df['purchase_date']).dt.to_period('M')

cat_summary = (df.groupby('category')
               .agg(Total_Sales=('Sales', 'sum'),
                   Avg_Order_Value=('Sales', 'mean'),
                   Transactions=('Sales', 'count'),
                   Total_Qty=('quantity', 'sum'))
               .sort_values('Total_Sales', ascending=False)
               .reset_index())

print(cat_summary)

# Optional: category × month trend
monthly = (df.groupby(['category', 'Month'])['Sales']
           .sum()
           .reset_index())
```

Explanation: Creating the Sales column (price × quantity) is the first essential transformation. groupby + named aggregation produces a compact category-level dashboard. Extracting Month enables time-series analysis of each category's revenue trajectory.

(b) Suggested Pivot-Table Structure

```
pivot = pd.pivot_table(df,
                       values='Sales',
                       index='category',
                       columns='Month',
                       aggfunc='sum',
                       fill_value=0,
                       margins=True)

print(pivot)
```

Recommended structure: rows = Category, columns = Month (or Region if available), values = sum of Sales. Adding margins supplies category totals and period totals. This matrix is ideal for a seaborn heatmap and for spotting seasonal category peaks.

(c) Matplotlib Plots for Transaction-Value Distribution

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

# 1. Histogram of Sales (log scale if heavily skewed)
axes[0].hist(df['Sales'], bins=30, color='steelblue', edgecolor='black')
axes[0].set_title('Sales Distribution')
axes[0].set_xlabel('Sales Value')

# 2. Box plot (overall or by category)
axes[1].boxplot(df['Sales'], vert=True)
axes[1].set_title('Sales Box Plot')

# 3. ECDF - cumulative view
from statsmodels.distributions.empirical_distribution import ECDF
ecdf = ECDF(df['Sales'])
axes[2].plot(ecdf.x, ecdf.y, color='darkgreen')
axes[2].set_title('ECDF of Sales')
axes[2].set_xlabel('Sales Value')
axes[2].set_ylabel('Proportion ≤ x')

plt.tight_layout()
plt.show()
```

Explanation: Histogram reveals shape and skewness; box plot highlights outliers and quartiles; ECDF answers quantile questions ("What % of orders exceed Rs.X?"). If Sales is heavily right-skewed, a log-transformed histogram is often more informative.

(d) Seaborn for Visualizing Relationships

```
# Price vs Quantity, coloured by category
sns.scatterplot(data=df, x='price', y='quantity', hue='category', alpha=0.6)
plt.title('Price vs Quantity by Category')
plt.show()

# Sales distribution by category
sns.boxplot(data=df, x='category', y='Sales', palette='Set2')
plt.xticks(rotation=45)
plt.title('Sales by Category')
plt.show()

# Correlation heatmap of numeric variables
sns.heatmap(df[['price', 'quantity', 'Sales']].corr(),
            annot=True, cmap='coolwarm', center=0)
plt.title('Numeric Feature Correlations')
plt.show()

# Time trend
sns.lineplot(data=df, x='purchase_date', y='Sales',
             estimator='sum', errorbar=None)
plt.title('Daily Total Sales Trend')
plt.xticks(rotation=45)
plt.show()
```

Seaborn's semantic mapping (hue, size, style) and statistical estimators make multi-variable relationship exploration concise and visually rich – ideal for the rapid iteration that EDA demands.

Question 12 (16 Marks) – Geographical / Pollution Data

Dataset: latitude, longitude, population, pollution. Tasks: Prepare for geo-viz; Map visualization approach; Colours & annotations for pollution; Combining geo-viz with other EDA plots.

(a) Preparing Data for Geographical Visualization

```
# Assume raw_df has columns: lat, lon, population, pollution
geo = raw_df.copy()

# 1. Type safety & range checks
geo['lat'] = pd.to_numeric(geo['lat'], errors='coerce')
geo['lon'] = pd.to_numeric(geo['lon'], errors='coerce')
geo = geo.dropna(subset=['lat', 'lon'])
geo = geo[(geo['lat'].between(-90,90)) & (geo['lon'].between(-180,180))]

# 2. Handle missing pollution / population
geo['pollution'] = geo['pollution'].fillna(geo['pollution'].median())
geo['population'] = geo['population'].fillna(0)

# 3. Derived columns for visual encoding
geo['pollution_level'] = pd.cut(geo['pollution'],
                                bins=[-np.inf, 50, 100, 200, np.inf],
                                labels=['Good', 'Moderate', 'Unhealthy', 'Hazardous'])

# 4. Optional: convert to GeoDataFrame (if geopandas installed)
# from geopandas import GeoDataFrame
# from shapely.geometry import Point
# gdf = GeoDataFrame(geo, geometry=[Point(xy) for xy in zip(geo.lon, geo.lat)],
#                    crs='EPSG:4326')
```

Explanation: Cleaning coordinates, imputing missing values, and creating a categorical pollution_level are essential pre-processing steps. A GeoDataFrame adds geometry awareness for advanced spatial operations, but a plain DataFrame is sufficient for most scatter-on-map plots.

(b) Appropriate Map Visualization

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(9, 7))
scatter = ax.scatter(geo['lon'], geo['lat'],
                    c=geo['pollution'],          # colour = pollution
                    s=geo['population']/500,      # size = population
                    cmap='RdYlGn_r', alpha=0.75,
                    edgecolors='k', linewidths=0.3)
plt.colorbar(scatter, ax=ax, label='Pollution Level')
ax.set_xlabel('Longitude')
ax.set_ylabel('Latitude')
ax.set_title('Geographic Distribution of Pollution (size ∝ population)')
plt.show()

# Interactive alternative (Folium / Plotly) is preferred for exploration
# import folium
# m = folium.Map(location=[geo.lat.mean(), geo.lon.mean()], zoom_start=10)
# for _, r in geo.iterrows():
#     folium.CircleMarker([r.lat, r.lon], radius=5,
#                         color='red' if r.pollution>100 else 'green',
#                         popup=f"Pop:{r.population}, Poll:{r.pollution}").add_to(m)
```

A scatter plot on geographic axes (or an interactive Folium/Plotly map) places each location correctly. Marker size encodes population; colour encodes pollution – two visual channels carrying two variables simultaneously.

(c) Colours & Annotations for Communicating Pollution

- A reversed Red-Yellow-Green colormap (RdYlGn_r) maps low pollution → green and high → red, matching intuitive traffic-light semantics.
- A colour-bar legend quantifies the continuous scale.
- Text annotations or hover tooltips display exact pollution values and location names.
- Threshold contours or specially highlighted markers can flag locations that exceed safe limits, drawing immediate attention for policy action.

(d) Combining Geographic Visualization with Other EDA Plots

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

# 1. Map (already shown conceptually)
# 2. Scatter: population vs pollution
sns.scatterplot(data=geo, x='population', y='pollution',
                hue='pollution_level', ax=axes[0])
axes[0].set_title('Population vs Pollution')

# 3. Distribution of pollution
sns.histplot(geo['pollution'], kde=True, ax=axes[1], color='crimson')
axes[1].set_title('Pollution Distribution')

# 4. Box plot by pollution category
sns.boxplot(data=geo, x='pollution_level', y='population', ax=axes[2])
axes[2].set_title('Population by Pollution Level')

plt.tight_layout()
plt.show()
```

Geographic maps answer “Where?”; statistical plots answer “How are the variables related?” and “What is the distribution?”. Placing them side-by-side (or linking them interactively) lets an analyst confirm that a spatial cluster of high pollution is also statistically extreme, or discover that high-population areas are not necessarily the most polluted. This multi-view approach is the hallmark of thorough exploratory analysis and supports evidence-based urban-planning decisions.

End of Complete 16-Mark Answers · All code is executable Python (pandas / matplotlib / seaborn)